Rapport de Projet

Système de Gestion de Pharmacie

Application Java avec Interface Graphique Swing

Pharmacy Manager

Projet de Développement Orienté Objet

Technologies utilisées :
Java SE, Swing, JDBC, MySQL

Réalisé par :
Kenza Benkaab
Khalil Jadlaoui

Année Universitaire 2025-2026

Table des matières

1 Introduction **4**
1.1 Contexte du Projet 4
1.2 Objectifs 4
1.3 Fonctionnalités Principales 4

2 Modèle Conceptuel **5**
2.1 Architecture du Système 5
2.2 Diagramme de Classes UML 5
2.2.1 Package Models 5
2.2.2 Package DAO 6
2.2.3 Package Services 6
2.3 Diagramme de Cas d'Utilisation 7
2.4 Modèle de Données 7
2.4.1 Schéma de la Base de Données 7

3 Code Source **9**
3.1 Package Models 9
3.1.1 Classe User (Exemple représentatif) 9
3.2 Package DAO 10
3.2.1 Classe BaseDAO (Exemple représentatif) 10
3.3 Package Services 12
3.3.1 Service d'Authentification (Exemple représentatif) 12
3.4 Package Utils 15
3.5 Package Controllers 15
3.6 Package Views 15
3.6.1 Fenêtre de Connexion (Exemple représentatif) 15
3.7 Classe Main 17

4 Captures d'Écran de l'Application **19**
4.1 Écran de Connexion 20
4.2 Tableau de Bord Gestionnaire 21
4.3 Tableau de Bord Pharmacien 21
4.4 Profils Utilisateurs 22
4.5 Gestion du Stock 22
4.6 Alerte Stock Critique 23
4.7 Enregistrement d'une Vente 23
4.8 Consultation des Ventes 24
4.9 Gestion des Clients 24
4.10 Historique du Stock 25

4.11 Rapports 25
4.12 Notifications Email 26
4.13 Structure du Projet 27

5 Conclusion **29**
5.1 Résumé du Projet 29
5.2 Technologies Utilisées 29
5.3 Points Forts de l'Application 29
5.4 Améliorations Possibles 30
5.5 Compétences Acquises 30

A Script SQL de Création de la Base de Données **31**

B Configuration du Projet **34**
B.1 Fichier module-info.java 34
B.2 Dépendances Requises 34
B.3 Instructions d'Installation 34

Table des figures

2.1 Architecture en couches de l'application 5
2.2 Diagramme de classes - Package Models 6
2.3 Diagramme de classes - Package DAO 6
2.4 Diagramme de classes - Package Services 7
2.5 Diagramme de cas d'utilisation 7

4.1 Interface de connexion 20
4.2 Tableau de bord du gestionnaire 21
4.3 Tableau de bord du pharmacien 21
4.4 Profils Pharmacien et Gestionnaire 22
4.5 Interface de gestion du stock 22
4.6 Alerte de stock critique 23
4.7 Interface d'enregistrement de vente 23
4.8 Liste des ventes 24
4.9 Interface de gestion des clients 24
4.10 Historique des mouvements de stock 25
4.11 Interface des rapports 25
4.12 Rapport du gestionnaire 26
4.13 Système de notifications 26
4.14 Boîte mail avec notifications 27
4.15 Structure du projet - Partie 1 27
4.16 Structure du projet - Partie 2 28

Chapitre 1

Introduction

1.1 Contexte du Projet

Ce projet consiste en la conception et le développement d'une application de gestion de pharmacie. L'application permet de gérer l'ensemble des opérations quotidiennes d'une pharmacie, incluant la gestion des médicaments, des ventes, des commandes et des utilisateurs.

1.2 Objectifs

Les objectifs principaux de ce projet sont :

- Développer une application Java complète avec interface graphique
- Implémenter l'architecture MVC (Modèle-Vue-Contrôleur)
- Utiliser le pattern DAO (Data Access Object) pour la persistance
- Gérer les transactions et la sécurité des données
- Fournir une interface utilisateur moderne et intuitive

1.3 Fonctionnalités Principales

L'application offre les fonctionnalités suivantes :

- **Authentification** : Connexion sécurisée pour pharmaciens et gestionnaires
- **Gestion du stock** : Suivi des médicaments, alertes de stock critique
- **Gestion des ventes** : Enregistrement et suivi des ventes
- **Gestion des commandes** : Commandes de réapprovisionnement
- **Gestion des clients** : Base de données des clients
- **Rapports** : Génération de rapports statistiques
- **Notifications par email** : Alertes automatiques
- **Export de données** : Export CSV et HTML

Chapitre 2

Modèle Conceptuel

2.1 Architecture du Système

L'application suit une architecture en couches basée sur le pattern MVC :

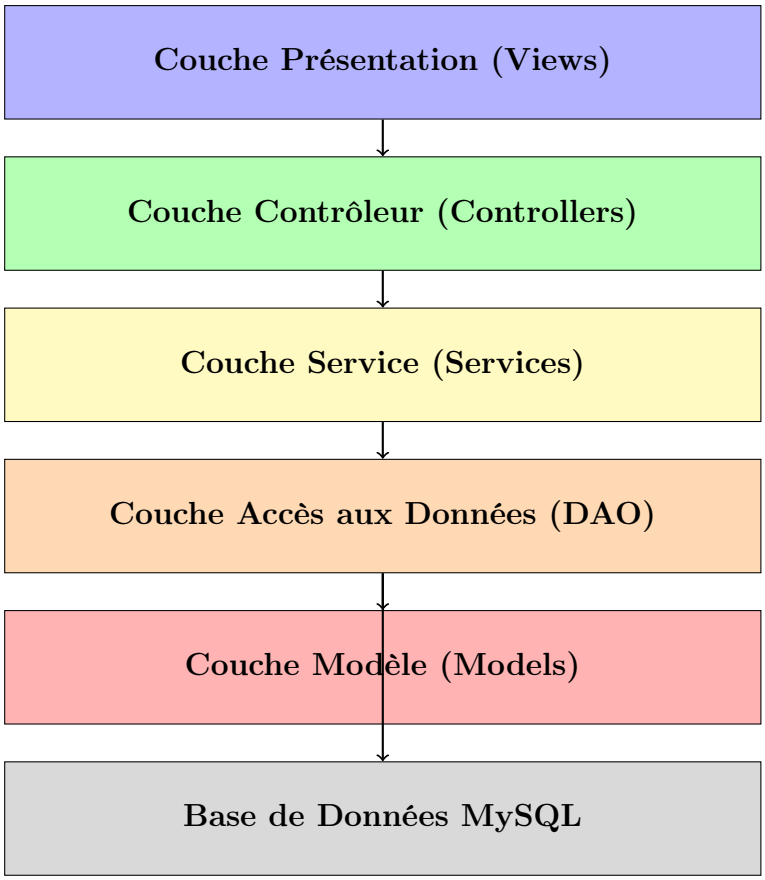

FIGURE 2.1 – Architecture en couches de l'application

2.2 Diagramme de Classes UML

2.2.1 Package Models

Le package `models` contient les entités métier de l'application :

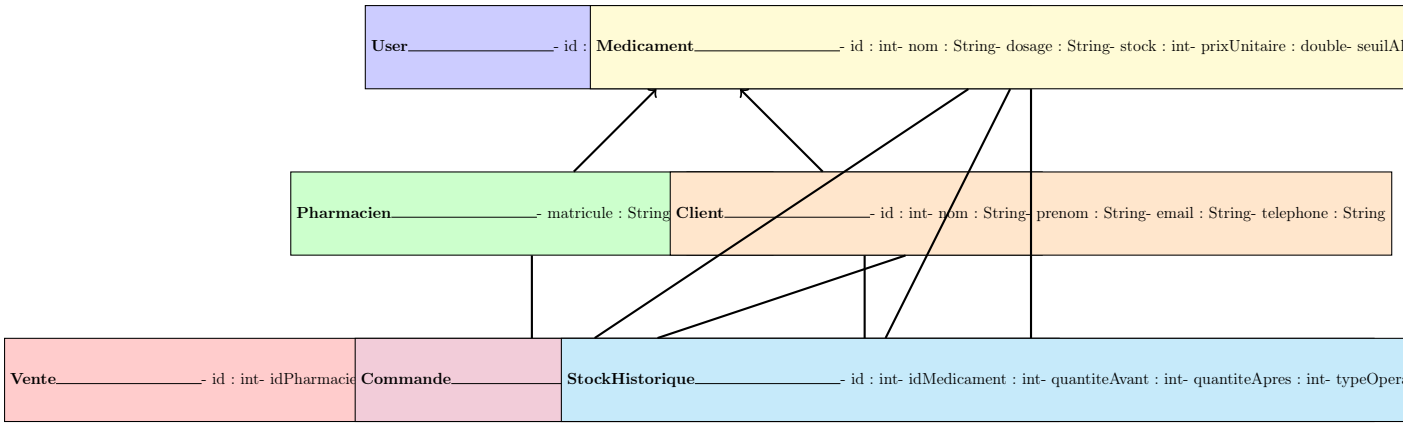

FIGURE 2.2 – Diagramme de classes - Package Models

2.2.2 Package DAO

Le package `dao` implémente le pattern Data Access Object :

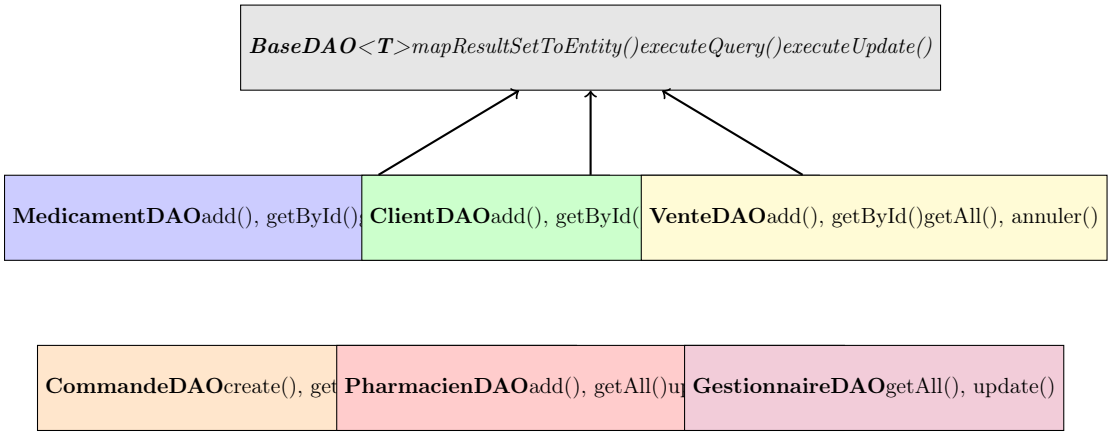

FIGURE 2.3 – Diagramme de classes - Package DAO

2.2.3 Package Services

Le package `services` contient la logique métier :

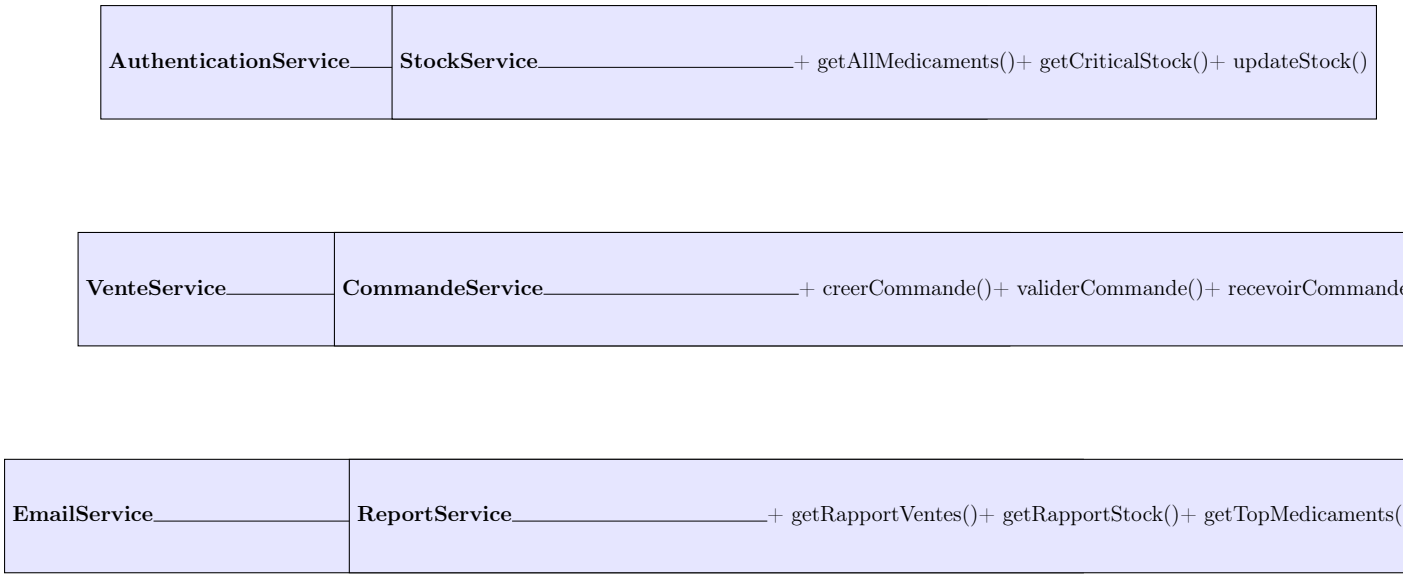

FIGURE 2.4 – Diagramme de classes - Package Services

2.3 Diagramme de Cas d'Utilisation

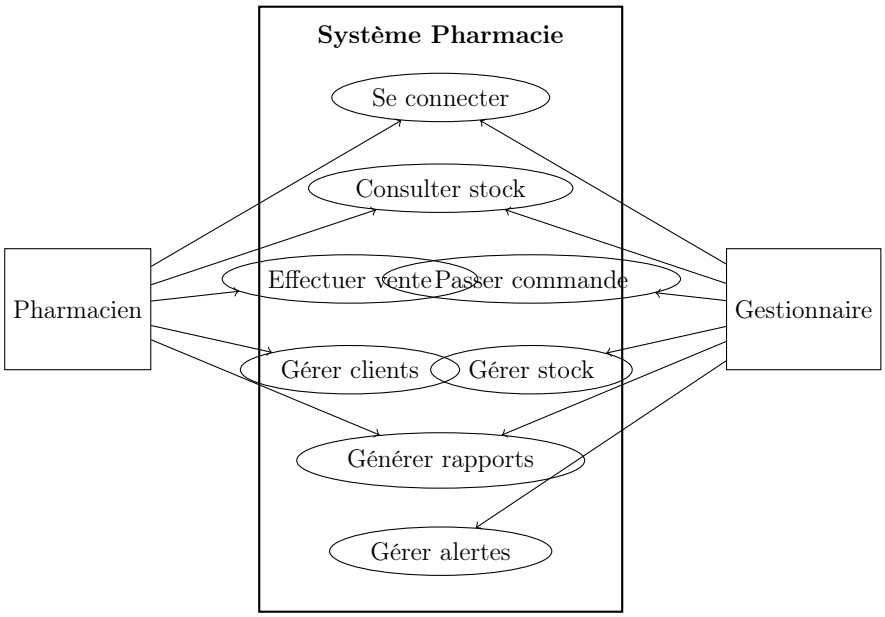

FIGURE 2.5 – Diagramme de cas d'utilisation

2.4 Modèle de Données

2.4.1 Schéma de la Base de Données

La base de données MySQL comprend les tables suivantes :

Colonne	Type	Contrainte	Description
id_medicament	INT	PRIMARY KEY, AUTO_INCREMENT	Identifiant unique
nom	VARCHAR(100)	NOT NULL	Nom du médicament
dosage	VARCHAR(50)		Dosage
stock	INT	DEFAULT 0	Quantité en stock
prix_unitaire	DECIMAL(10,2)		Prix unitaire
seuil_alerte	INT	DEFAULT 10	Seuil d'alerte
description	TEXT		Description

TABLE 2.1 – Structure de la table `medicament`

Colonne	Type	Contrainte	Description
id_vente	INT	PRIMARY KEY	Identifiant unique
id_pharmacien	INT	FOREIGN KEY	Référence pharmacien
id_client	INT	FOREIGN KEY	Référence client
id_medicament	INT	FOREIGN KEY	Référence médicament
quantite	INT	NOT NULL	Quantité vendue
prix_total	DECIMAL(10,2)		Prix total
date_vente	TIMESTAMP	DEFAULT NOW()	Date de vente
statut	VARCHAR(20)	DEFAULT 'valide'	Statut

TABLE 2.2 – Structure de la table `vente`

Colonne	Type	Contrainte	Description
id_commande	INT	PRIMARY KEY	Identifiant unique
id_gestionnaire	INT	FOREIGN KEY	Référence gestionnaire
id_medicament	INT	FOREIGN KEY	Référence médicament
quantite	INT	NOT NULL	Quantité commandée
date_commande	TIMESTAMP	DEFAULT NOW()	Date de commande
statut	VARCHAR(20)	DEFAULT 'en_attente'	Statut
date_reception	TIMESTAMP		Date de réception

TABLE 2.3 – Structure de la table `commande`

Chapitre 3

Code Source

Cette section présente l'intégralité du code source de l'application, organisé par package.

3.1 Package Models

Le package `models` contient les classes représentant les entités métier. Les principales classes sont : `User`, `Pharmacien`, `Gestionnaire`, `Medicament`, `Client`, `Vente`, `Commande` et `StockHistorique`.

Note : Pour des raisons de concision, seule la classe `User` est présentée comme exemple. Les autres classes suivent un pattern similaire avec leurs attributs spécifiques.

3.1.1 Classe User (Exemple représentatif)

```java
package models;

/**
 * Classe representant un utilisateur generique
 * Classe de base pour Pharmacien et Gestionnaire
 */
public class User {
    private int id;
    private String nom;
    private String prenom;
    private String login;
    private String role;

    // Constructeurs
    public User() {}

    // Getters et Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }

    public String getPrenom() { return prenom; }
    public void setPrenom(String prenom) { this.prenom = prenom; }

```

```
    public String getLogin() { return login; }
    public void setLogin(String login) { this.login = login; }

    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
}
```

Listing 3.1 – User.java - Classe représentant un utilisateur

Autres classes du package Models :

- `Pharmacien.java` - Hérite de User, attributs : matricule, pwd
- `Gestionnaire.java` - Hérite de User, attributs : pwd
- `Medicament.java` - Attributs : id, nom, dosage, stock, prixUnitaire, seuilAlerte, description
- `Client.java` - Attributs : id, nom, prenom, email, adresse, telephone
- `Vente.java` - Attributs : id, idPharmacien, idClient, idMedicament, quantite, prixTotal, dateVente, statut
- `Commande.java` - Attributs : id, idGestionnaire, idMedicament, quantite, dateCommande, statut, dateReception
- `StockHistorique.java` - Attributs : id, idMedicament, quantiteAvant, quantiteApres, typeOperation, dateOperation

3.2 Package DAO

Le package `dao` implémente le pattern Data Access Object pour l'accès aux données.

Note : Pour des raisons de concision, seule la classe `BaseDAO` est présentée comme exemple. Les autres classes DAO (`MedicamentDAO`, `VenteDAO`, `CommandeDAO`, etc.) suivent un pattern similaire.

3.2.1 Classe BaseDAO (Exemple représentatif)

```
package dao;

import utils.DatabaseManager;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * Classe abstraite de base pour tous les DAO
 * Implemente les operations CRUD generiques
 * @param <T> Le type d'entite geree par le DAO
 */
public abstract class BaseDAO<T> {
    protected DatabaseManager dbManager;

    public BaseDAO() {
        this.dbManager = DatabaseManager.getInstance();
    }

    /**
```

```java
     * Methode abstraite pour mapper un ResultSet vers une entite
     */
    protected abstract T mapResultSetToEntity(ResultSet rs)
        throws SQLException;

    /**
     * Execute une requete SELECT et retourne une liste
     */
    protected List<T> executeQuery(String sql, Object... params) {
        List<T> results = new ArrayList<>();

        try (Connection conn = dbManager.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            setParameters(pstmt, params);

            try (ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                    results.add(mapResultSetToEntity(rs));
                }
            }

        } catch (SQLException e) {
            handleSQLException("Erreur lors de l'execution", e, sql);
        }

        return results;
    }

    /**
     * Execute une requete SELECT et retourne un seul resultat
     */
    protected T executeQuerySingle(String sql, Object... params) {
        List<T> results = executeQuery(sql, params);
        return results.isEmpty() ? null : results.get(0);
    }

    /**
     * Execute une requete INSERT/UPDATE/DELETE
     */
    protected int executeUpdate(String sql, Object... params) {
        try (Connection conn = dbManager.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(
                 sql, Statement.RETURN_GENERATED_KEYS)) {

            setParameters(pstmt, params);
            int affectedRows = pstmt.executeUpdate();

            // Recuperer l'ID genere si INSERT
            if (affectedRows > 0 &&
                sql.trim().toUpperCase().startsWith("INSERT")) {
                try (ResultSet generatedKeys =
                     pstmt.getGeneratedKeys()) {
                    if (generatedKeys.next()) {
                        return generatedKeys.getInt(1);
                    }
                }
            }
```

```java

            return affectedRows;

        } catch (SQLException e) {
            handleSQLException("Erreur lors de la mise a jour", e, sql);
            return 0;
        }
    }

    /**
     * Configure les parametres d'une requete preparee
     */
    private void setParameters(PreparedStatement pstmt,
                               Object... params) throws SQLException {
        for (int i = 0; i < params.length; i++) {
            pstmt.setObject(i + 1, params[i]);
        }
    }

    /**
     * Gere les exceptions SQL
     */
    private void handleSQLException(String message, SQLException e,
                                    String sql) {
        System.err.println("Erreur: " + message);
        System.err.println("Requete: " + sql);
        System.err.println("Message: " + e.getMessage());
    }
}
```

Listing 3.2 – BaseDAO.java - Classe abstraite de base pour les DAO

Autres classes DAO :

- `MedicamentDAO.java` - Opérations CRUD pour les médicaments (add, getById, getAll, update, delete, diminuerStock, getStockCritique)
- `VenteDAO.java` - Opérations sur les ventes (add, getAll, annuler, getByPeriode)
- `CommandeDAO.java` - Opérations sur les commandes (add, getAll, valider, recevoir, annuler)
- `ClientDAO.java` - Opérations sur les clients (add, getById, getAll, update, delete)
- `PharmacienDAO.java` - Opérations sur les pharmaciens
- `GestionnaireDAO.java` - Opérations sur les gestionnaires
- `StockHistoriqueDAO.java` - Historique des mouvements de stock

3.3 Package Services

Le package `services` contient la logique métier de l'application.

Note : Pour des raisons de concision, seul le service `AuthenticationService` est présenté comme exemple.

3.3.1 Service d'Authentification (Exemple représentatif)

```java
package services;

import dao.PharmacienDAO;
import dao.GestionnaireDAO;
import models.User;
import models.Pharmacien;
import models.Gestionnaire;
import java.util.List;

/**
 * Service d'authentification
 * Gere la connexion des utilisateurs
 * Utilise le pattern Singleton
 */
public class AuthenticationService {

    private static AuthenticationService instance;
    private final PharmacienDAO pharmacienDAO;
    private final GestionnaireDAO gestionnaireDAO;

    private AuthenticationService() {
        this.pharmacienDAO = new PharmacienDAO();
        this.gestionnaireDAO = new GestionnaireDAO();
    }

    public static synchronized AuthenticationService getInstance() {
        if (instance == null) {
            instance = new AuthenticationService();
        }
        return instance;
    }

    /**
     * Authentifie un utilisateur
     * @param login Le nom d'utilisateur
     * @param password Le mot de passe
     * @param role Le role (PHARMACIEN ou GESTIONNAIRE)
     * @return L'objet User si authentification reussie, null sinon
     */
    public User login(String login, String password, String role) {
        System.out.println("=== AUTHENTIFICATION ===");
        System.out.println("Login: " + login);
        System.out.println("Role: " + role);

        try {
            if ("GESTIONNAIRE".equals(role)) {
                List<Gestionnaire> gestionnaires =
                    gestionnaireDAO.getAll();

                for (Gestionnaire g : gestionnaires) {
                    if (login.equals(g.getLogin()) &&
                        password.equals(g.getPwd())) {
                        System.out.println("Gestionnaire trouve: " +
                                           g.getNom());

                        User user = new User();
                        user.setId(g.getId());
```

```
                        user.setNom(g.getNom());
                        user.setPrenom(g.getPrenom());
                        user.setLogin(g.getLogin());
                        user.setRole("GESTIONNAIRE");
                        return user;
                    }
                }

            } else if ("PHARMACIEN".equals(role)) {
                List<Pharmacien> pharmaciens = pharmacienDAO.getAll();

                for (Pharmacien p : pharmaciens) {
                    if (login.equals(p.getLogin()) &&
                        password.equals(p.getPwd())) {
                        System.out.println("Pharmacien trouve: " +
                                           p.getNom());

                        User user = new User();
                        user.setId(p.getId());
                        user.setNom(p.getNom());
                        user.setPrenom(p.getPrenom());
                        user.setLogin(p.getLogin());
                        user.setRole("PHARMACIEN");
                        return user;
                    }
                }
            }

            return null;

        } catch (Exception e) {
            System.err.println("Erreur authentification: " +
                               e.getMessage());
            return null;
        }
    }

    public void logout() {
        // Deconnexion
    }
}
```

Listing 3.3 – AuthenticationService.java - Service d'authentification

Autres services du package Services :

- `VenteService.java` - Logique métier des ventes (enregistrerVente, annulerVente, getAllVentes)
- `StockService.java` - Gestion du stock (ajouterMedicament, calculerValeurStock, getCriticalStock)
- `CommandeService.java` - Gestion des commandes (creerCommande, validerCommande, recevoirCommande)
- `EmailService.java` - Envoi de notifications (sendStockAlert, sendWeeklyReport)
- `ReportService.java` - Génération de rapports (getRapportVentes, getTopMedicaments)
- `ExportService.java` - Export de données (exportToCSV, exportToHTML)

3.4 Package Utils

Le package `utils` contient les classes utilitaires :

- `DatabaseConnection.java` - Connexion à MySQL (URL : jdbc :mysql ://localhost :3300/pharmacie_db)
- `DatabaseManager.java` - Pool de connexions (5 connexions)
- `TransactionManager.java` - Gestion des transactions avec commit/rollback
- `LoggerUtil.java` - Journalisation des événements

3.5 Package Controllers

Le package `controllers` contient :

- `LoginController.java` - Contrôle la connexion et redirige vers le dashboard approprié

3.6 Package Views

Le package `views` contient les interfaces graphiques Swing.

Note : Pour des raisons de concision, seule la classe `LoginFrame` est présentée comme exemple.

3.6.1 Fenêtre de Connexion (Exemple représentatif)

```java
package views;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

/**
 * Fenetre de connexion - Gestion de Pharmacie
 * Interface moderne avec style professionnel
 */
public class LoginFrame extends JFrame {

    // Couleurs
    private static final Color BG_MAIN = new Color(245, 247, 250);
    private static final Color PRIMARY = new Color(33, 150, 243);
    private static final Color TEXT_MAIN = new Color(55, 55, 55);
    private static final Color ERROR = new Color(211, 47, 47);
    private static final Color SUCCESS = new Color(46, 125, 50);

    // Composants
    private JTextField txtUsername;
    private JPasswordField txtPassword;
    private JComboBox<String> cmbUserType;
    private JButton btnLogin;
    private JButton btnCancel;
    private JLabel lblMessage;
```

```java

    // Controleur
    private controllers.LoginController controller;

    public LoginFrame() {
        initFrame();
        initComponents();
        buildLayout();

        // Initialiser le controleur
        this.controller = new controllers.LoginController(this);

        // Configurer les listeners
        setupEventListeners();

        setVisible(true);
    }

    private void initFrame() {
        setTitle("Gestion de Pharmacie - Connexion");
        setSize(520, 750);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setResizable(false);
        getContentPane().setBackground(BG_MAIN);
    }

    private void initComponents() {
        txtUsername = new JTextField();
        txtPassword = new JPasswordField();
        cmbUserType = new JComboBox<>(
            new String[]{"Pharmacien", "Gestionnaire"});

        styleField(txtUsername);
        styleField(txtPassword);
        styleField(cmbUserType);

        btnLogin = createPrimaryButton("Connexion");
        btnCancel = createSecondaryButton("Annuler");

        lblMessage = new JLabel(" ", SwingConstants.CENTER);
        lblMessage.setFont(new Font("Segoe UI", Font.PLAIN, 13));
    }

    private void setupEventListeners() {
        btnLogin.addActionListener(e -> {
            String username = txtUsername.getText().trim();
            String password = new String(txtPassword.getPassword());
            String userType = (String) cmbUserType.getSelectedItem();

            controller.handleLogin(username, password, userType);
        });

        btnCancel.addActionListener(e -> {
            controller.handleCancel();
        });

        // Entree pour login
```

```
        txtPassword.addActionListener(e -> {
            btnLogin.doClick();
        });
    }

    // Methodes publiques pour le controleur
    public void showError(String msg) {
        lblMessage.setText(msg);
        lblMessage.setForeground(ERROR);
    }

    public void showSuccess(String msg) {
        lblMessage.setText(msg);
        lblMessage.setForeground(SUCCESS);
    }

    public void clearFields() {
        txtUsername.setText("");
        txtPassword.setText("");
        cmbUserType.setSelectedIndex(0);
    }

    public void setLoading(boolean loading) {
        txtUsername.setEnabled(!loading);
        txtPassword.setEnabled(!loading);
        btnLogin.setEnabled(!loading);

        if (loading) {
            btnLogin.setText("Connexion...");
        } else {
            btnLogin.setText("Connexion");
        }
    }
}
```

Listing 3.4 – LoginFrame.java - Fenêtre de connexion

Autres classes du package Views :

- `GestionnaireDashboard.java` - Tableau de bord du gestionnaire (gestion stock, commandes, rapports)
- `PharmacienDashboard.java` - Tableau de bord du pharmacien (ventes, clients, consultation)
- `StockPanel.java` - Panel de gestion du stock
- `VentePanel.java` - Panel d'enregistrement des ventes
- `CommandePanel.java` - Panel de gestion des commandes
- `StockHistoriquePanel.java` - Panel de l'historique du stock
- `PharmacienFormDialog.java` - Dialogue de création de pharmacien

3.7 Classe Main

```
package main;

import javax.swing.*;
```

```java
import views.LoginFrame;
import utils.DatabaseManager;
import java.sql.Connection;

/**
 * Classe principale de l'application Pharmacy Manager
 * Point d'entree avec initialisation BD et interface
 */
public class Main {

    public static void main(String[] args) {
        // Configuration du look and feel
        setLookAndFeel();

        // Initialisation de la base de donnees
        if (!initializeDatabase()) {
            System.err.println("Impossible de demarrer sans BD");
            System.exit(1);
            return;
        }

        // Lancement de l'application
        SwingUtilities.invokeLater(() -> {
            LoginFrame loginFrame = new LoginFrame();
            loginFrame.setVisible(true);
        });
    }

    private static void setLookAndFeel() {
        try {
            UIManager.setLookAndFeel(
                UIManager.getSystemLookAndFeelClassName());
        } catch (Exception e) {
            System.err.println("Erreur Look and Feel: " + e.getMessage()
    );
        }
    }

    private static boolean initializeDatabase() {
        try {
            DatabaseManager dbManager = DatabaseManager.getInstance();
            Connection conn = dbManager.getConnection();
            return conn != null && !conn.isClosed();
        } catch (Exception e) {
            System.err.println("Erreur BD: " + e.getMessage());
            return false;
        }
    }
}
```

Listing 3.5 – Main.java - Point d'entrée de l'application

Chapitre 4

Captures d'Écran de l'Application

Cette section présente les captures d'écran de l'application en fonctionnement.

4.1 Écran de Connexion

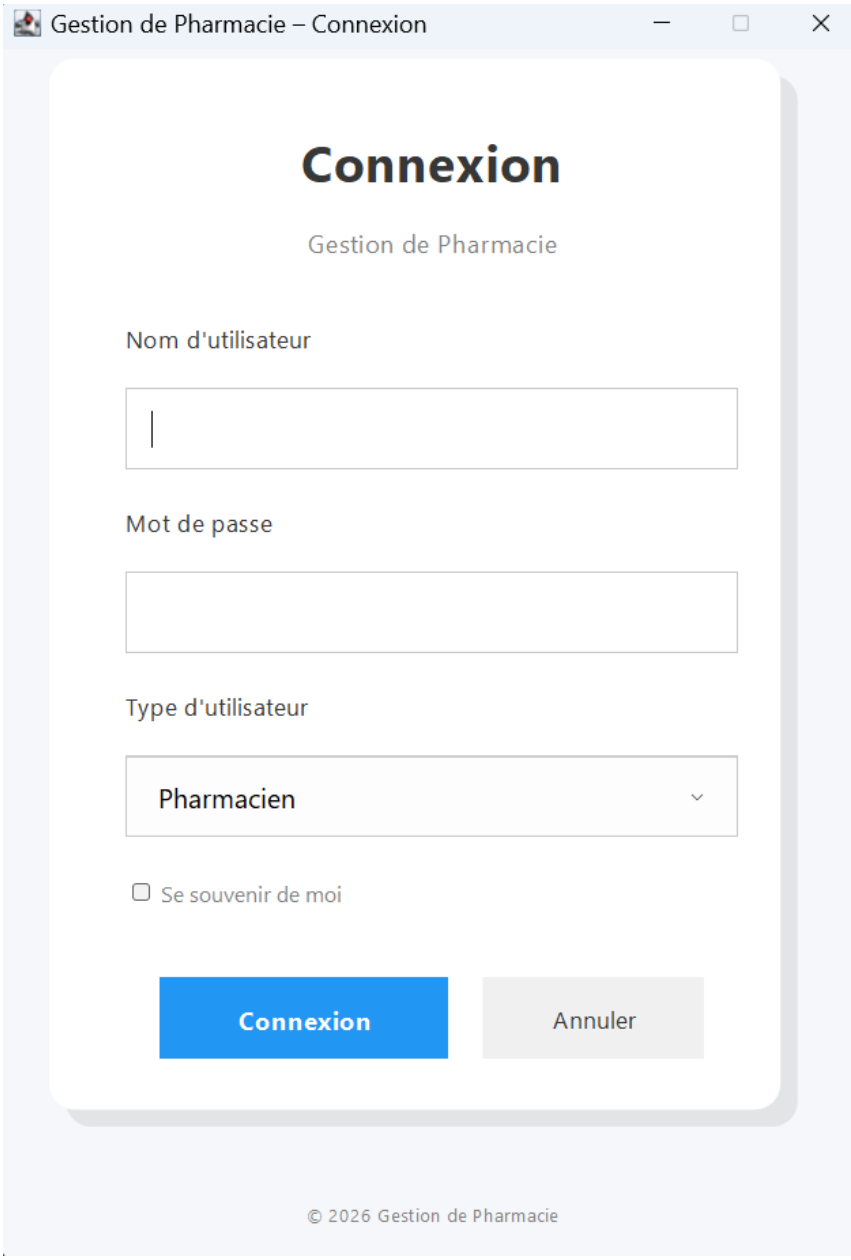

FIGURE 4.1 – Interface de connexion

Description : L'écran de connexion permet aux utilisateurs de s'authentifier en choisissant leur rôle (Pharmacien ou Gestionnaire) et en saisissant leurs identifiants.

4.2 Tableau de Bord Gestionnaire

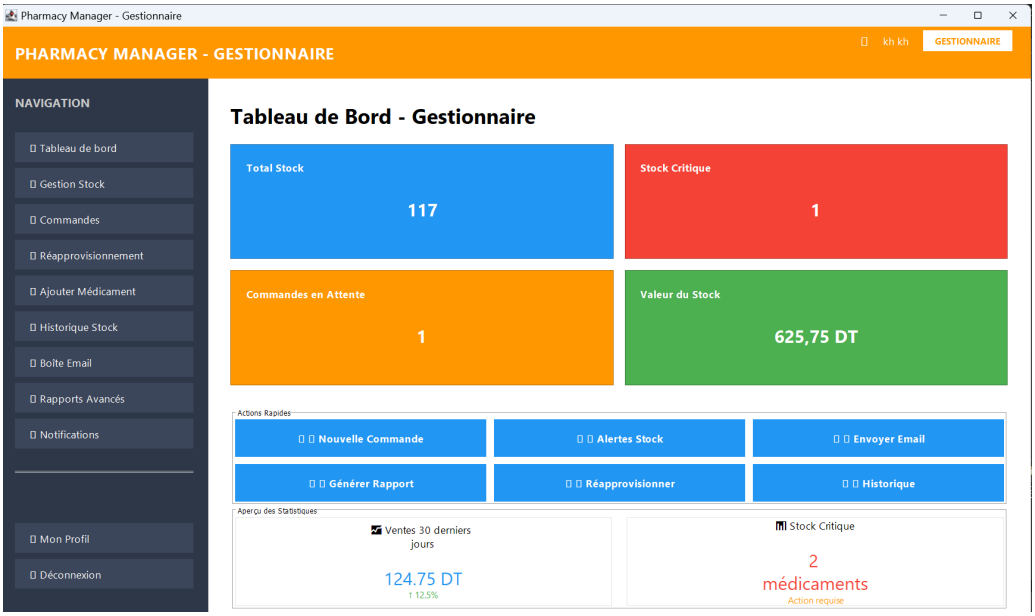

FIGURE 4.2 – Tableau de bord du gestionnaire

Description : Le tableau de bord du gestionnaire affiche les statistiques du stock, les commandes en attente et la navigation vers les différentes fonctionnalités.

4.3 Tableau de Bord Pharmacien

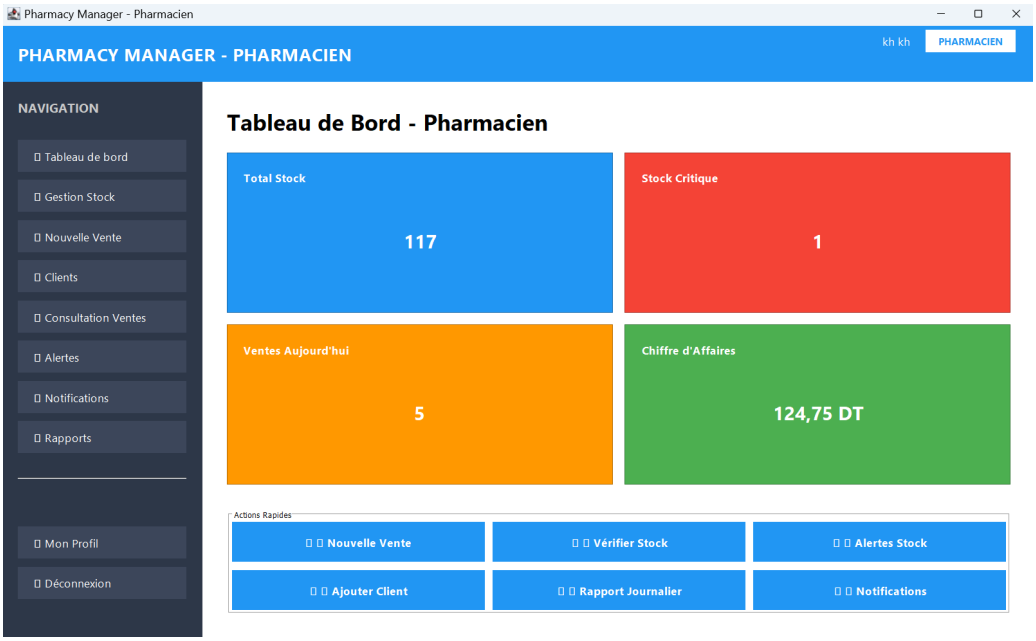

FIGURE 4.3 – Tableau de bord du pharmacien

Description : Le tableau de bord du pharmacien permet la visualisation des ventes du jour et l'accès rapide aux fonctionnalités.

4.4 Profils Utilisateurs

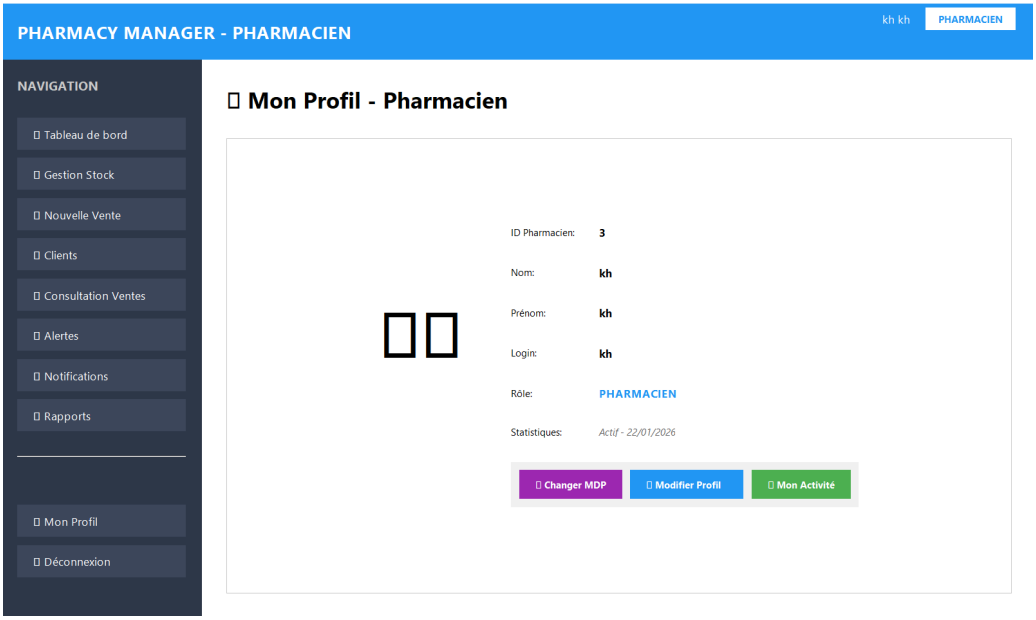

FIGURE 4.4 – Profils Pharmacien et Gestionnaire

4.5 Gestion du Stock

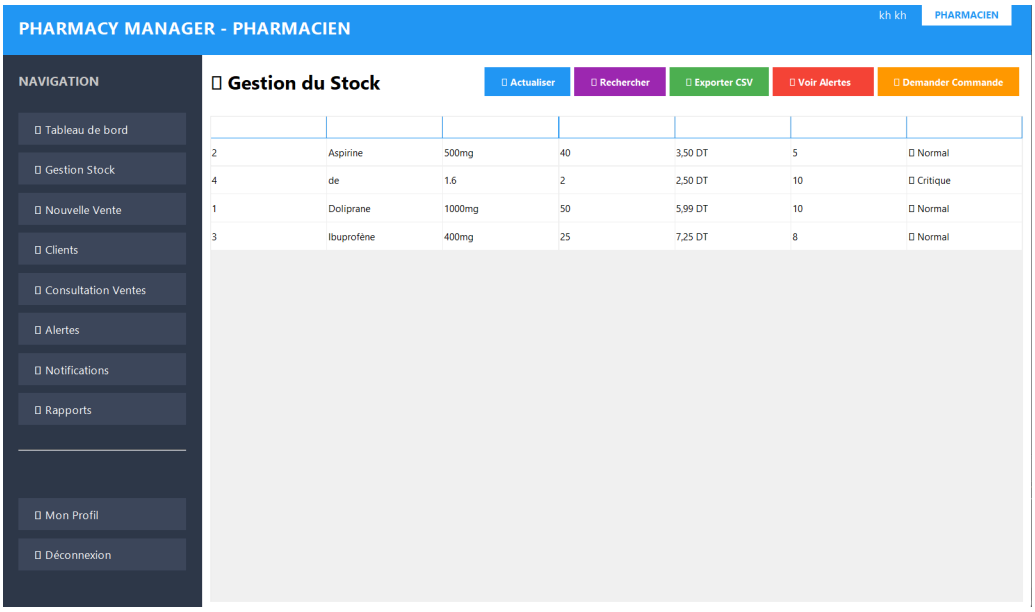

FIGURE 4.5 – Interface de gestion du stock

Description : L'interface de gestion du stock affiche la liste des médicaments avec les quantités et les indicateurs de stock critique.

4.6 Alerte Stock Critique

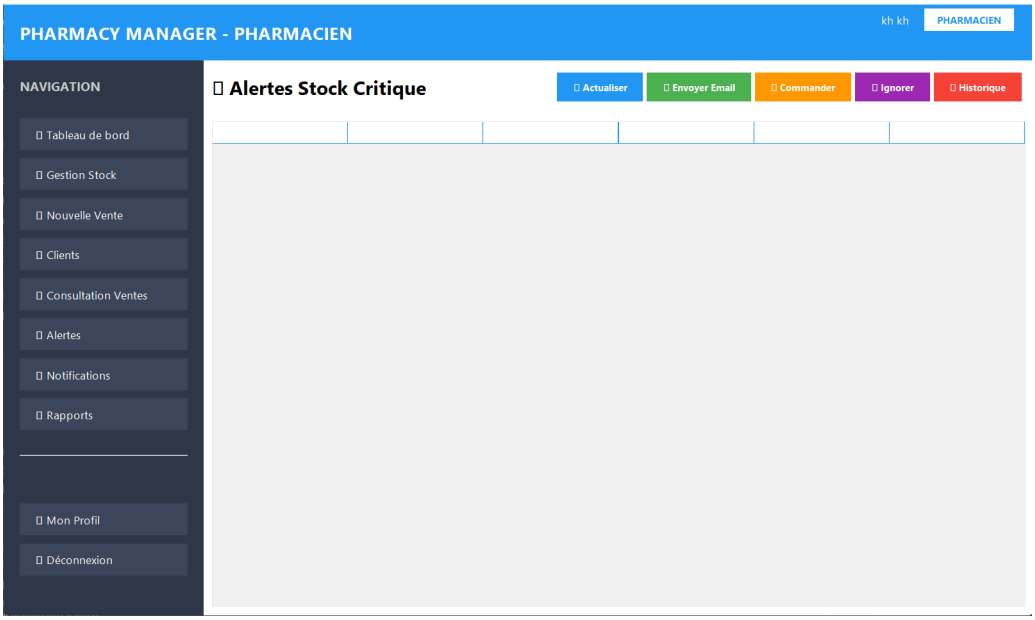

FIGURE 4.6 – Alerte de stock critique

4.7 Enregistrement d'une Vente

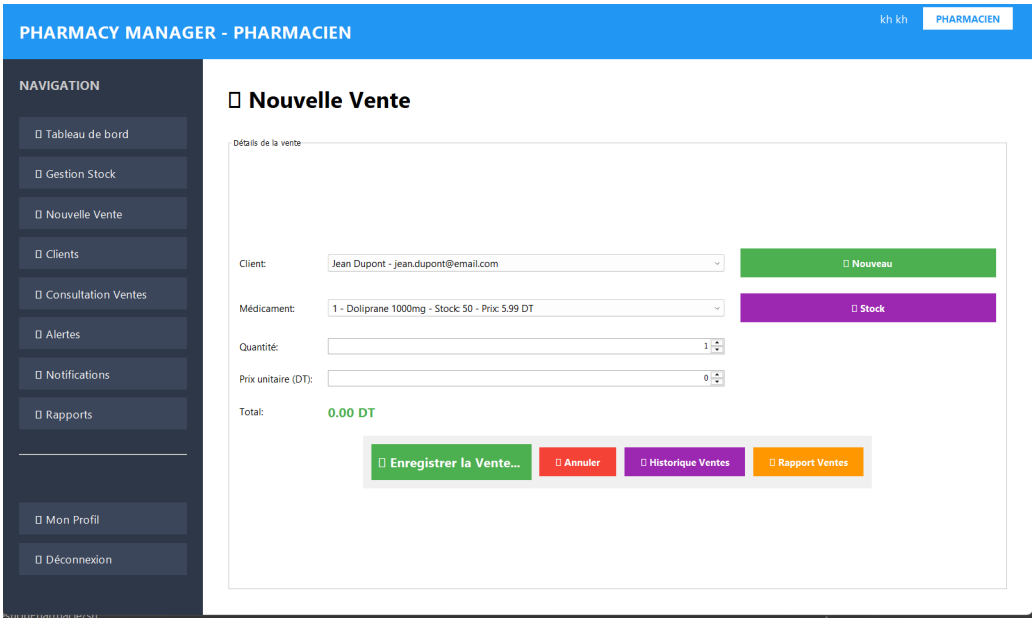

FIGURE 4.7 – Interface d'enregistrement de vente

Description : Le formulaire de vente permet la sélection du client et du médicament avec calcul automatique du prix total.

4.8 Consultation des Ventes

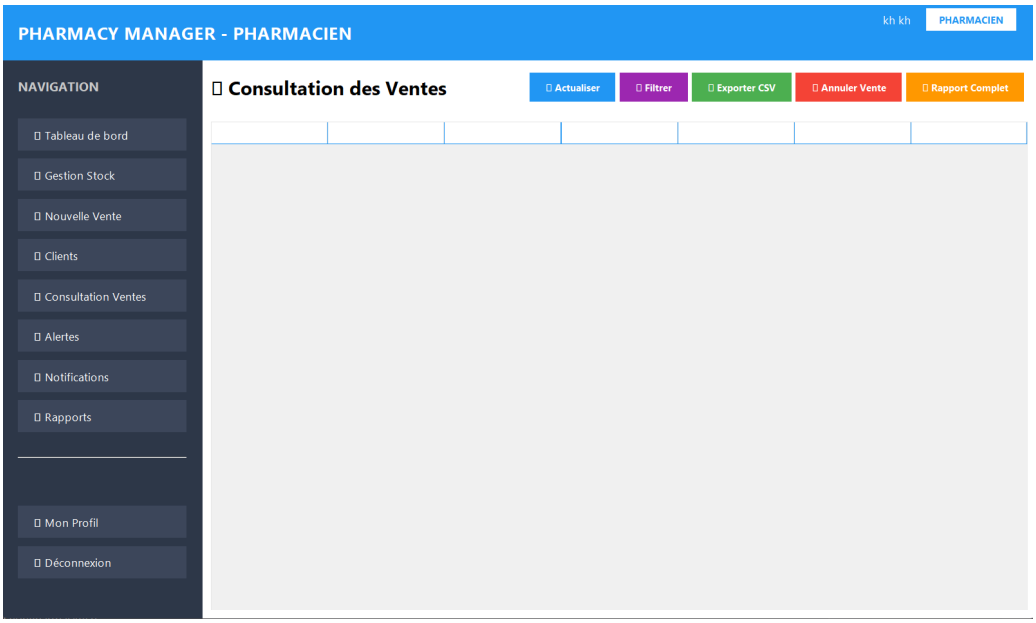

FIGURE 4.8 – Liste des ventes

4.9 Gestion des Clients

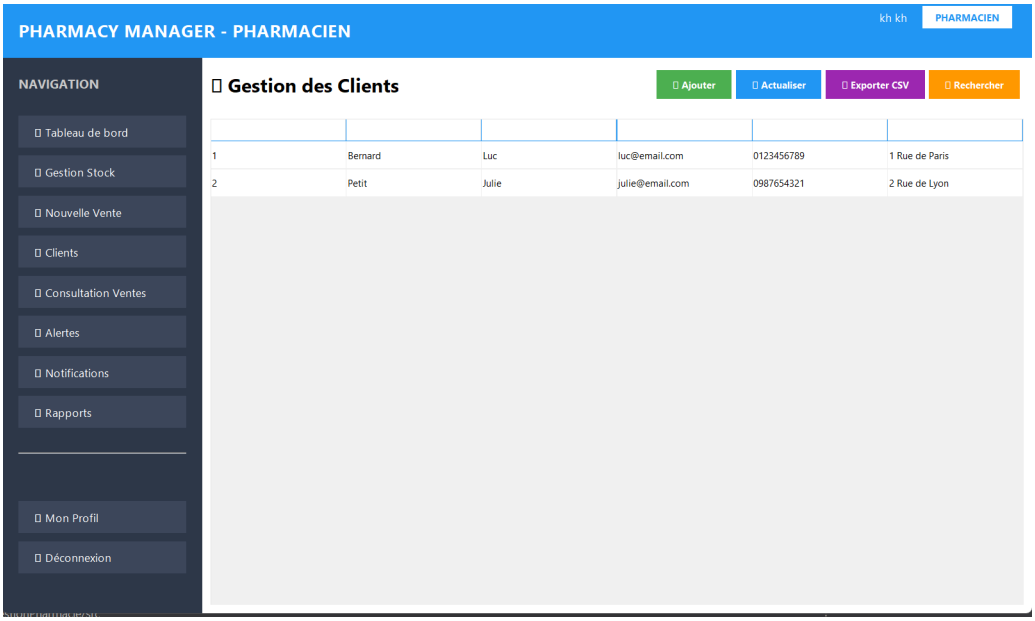

FIGURE 4.9 – Interface de gestion des clients

4.10 Historique du Stock

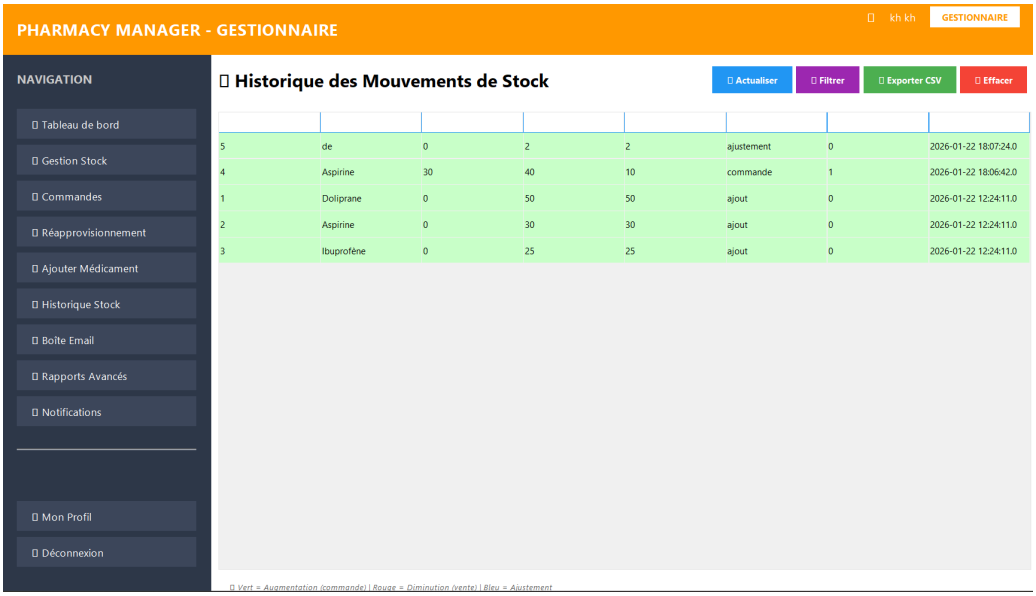

FIGURE 4.10 – Historique des mouvements de stock

Description : L'historique affiche tous les mouvements de stock (ventes, commandes, ajustements) avec les quantités avant et après chaque opération.

4.11 Rapports

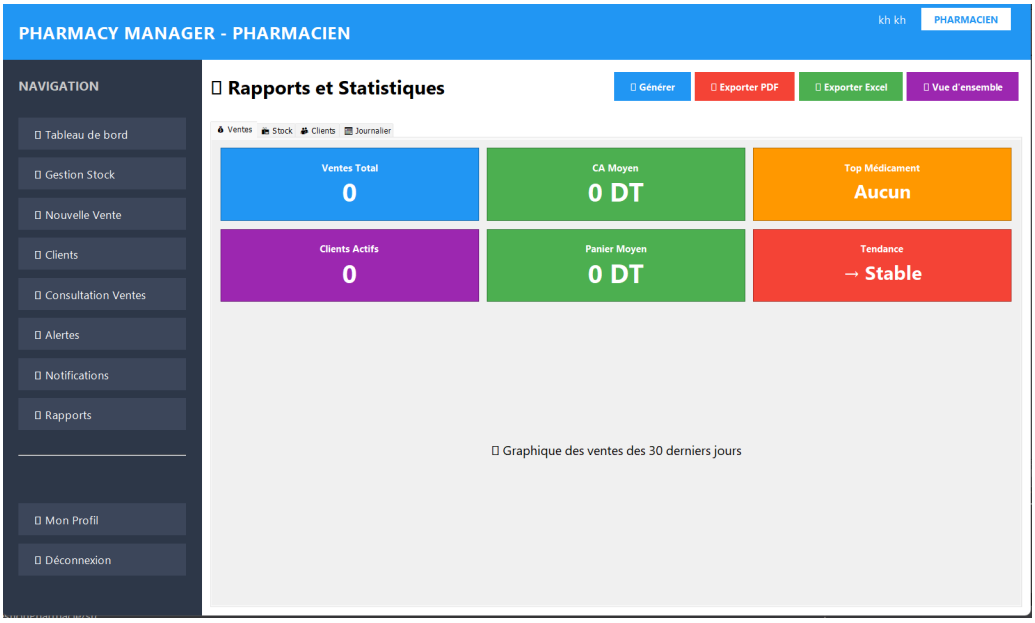

FIGURE 4.11 – Interface des rapports

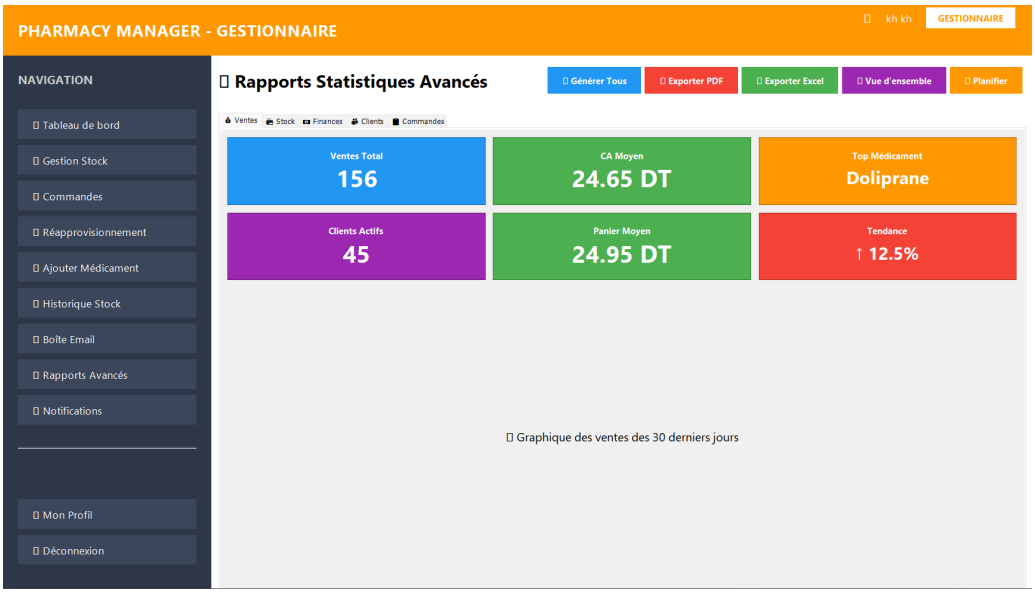

FIGURE 4.12 – Rapport du gestionnaire

4.12 Notifications Email

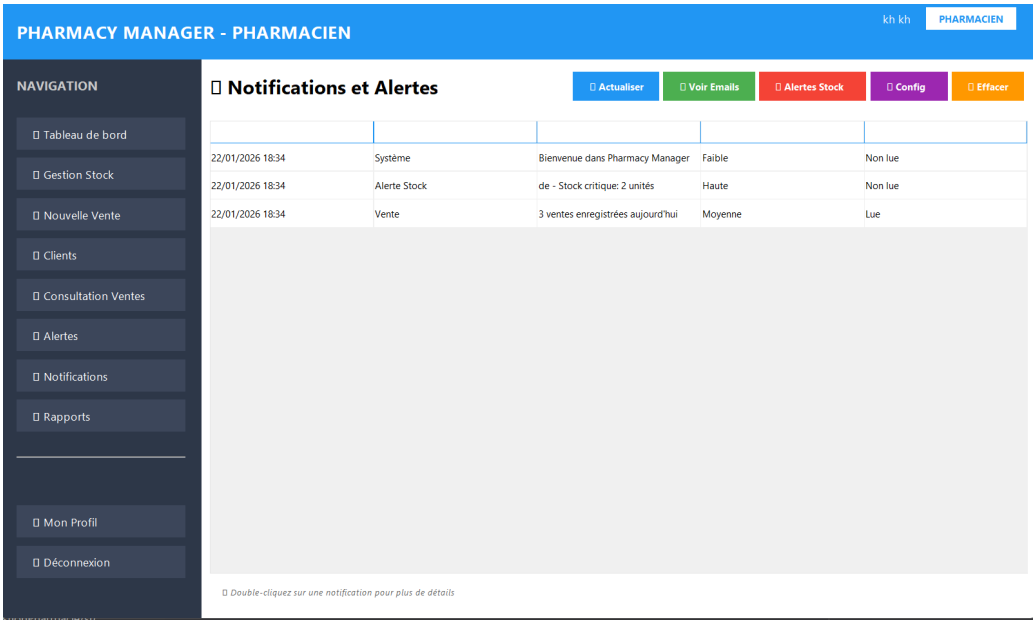

FIGURE 4.13 – Système de notifications

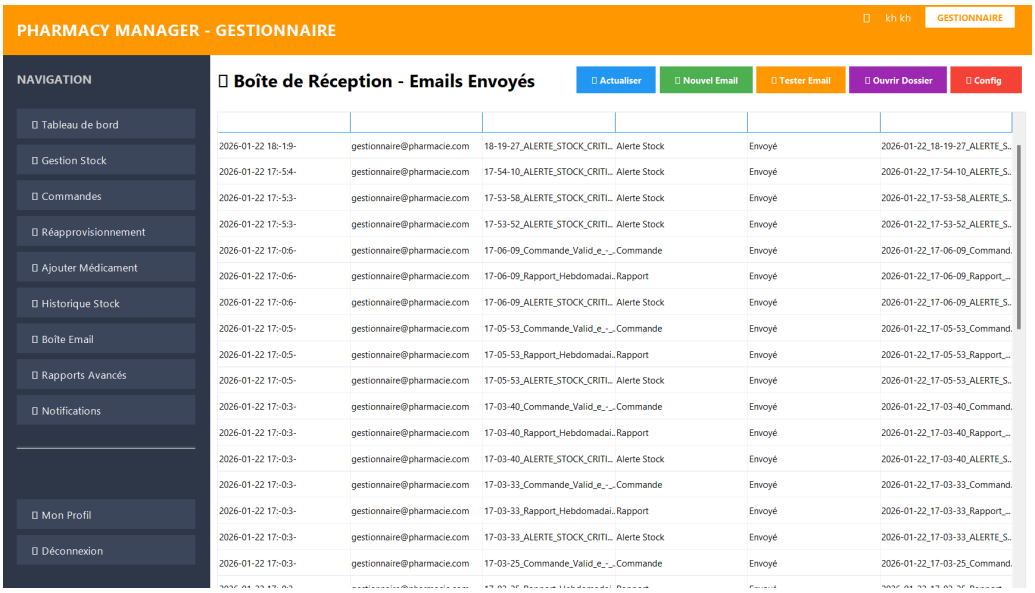

FIGURE 4.14 – Boîte mail avec notifications

4.13 Structure du Projet

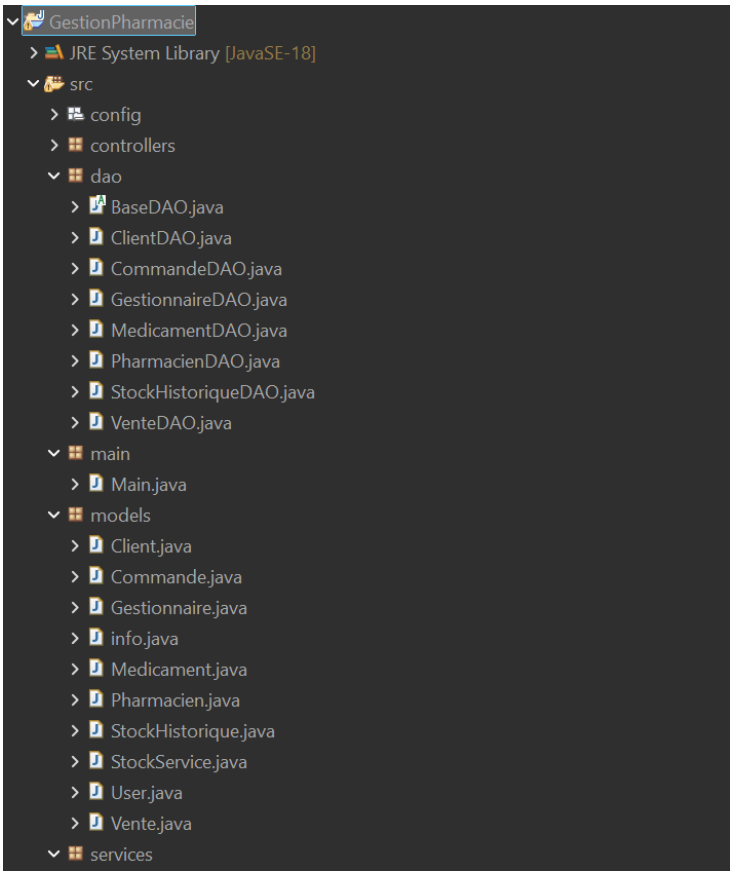

FIGURE 4.15 – Structure du projet - Partie 1

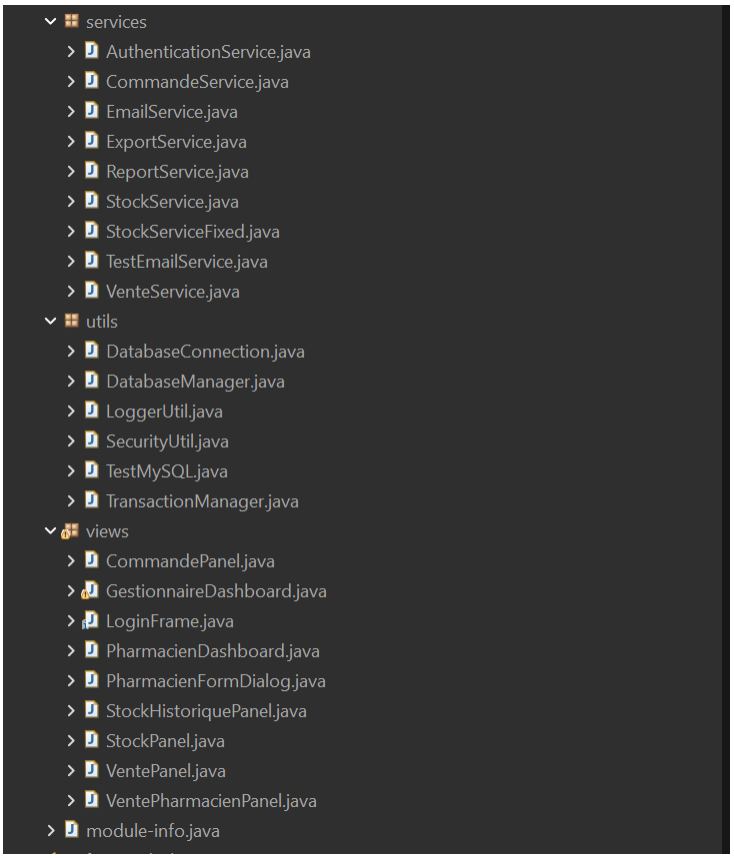

FIGURE 4.16 – Structure du projet - Partie 2

Chapitre 5

Conclusion

5.1 Résumé du Projet

Ce projet a permis de développer une application complète de gestion de pharmacie en Java, intégrant :

— Une architecture MVC bien structurée
— Le pattern DAO pour l'accès aux données
— Une interface graphique moderne avec Swing
— La gestion des transactions pour l'intégrité des données
— Un système de notification par email
— Des fonctionnalités d'export de données

5.2 Technologies Utilisées

— **Langage** : Java SE 11+
— **Interface graphique** : Java Swing
— **Base de données** : MySQL
— **Connecteur** : MySQL Connector/J (JDBC)
— **Architecture** : MVC avec pattern DAO
— **Gestion des transactions** : TransactionManager personnalisé

5.3 Points Forts de l'Application

1. **Séparation des responsabilités** : L'architecture en couches permet une maintenance facile
2. **Réutilisabilité** : Les classes DAO et services sont réutilisables
3. **Sécurité** : Gestion des transactions et validation des données
4. **Ergonomie** : Interface utilisateur intuitive et moderne
5. **Traçabilité** : Historique complet des mouvements de stock

5.4 Améliorations Possibles

Pour les versions futures, les améliorations suivantes pourraient être envisagées :

— Implémentation d'un système de sauvegarde automatique
— Ajout de graphiques statistiques
— Intégration d'un vrai serveur SMTP pour les emails
— Application mobile compagnon
— Système de codes-barres pour les médicaments
— Gestion multi-pharmacies

5.5 Compétences Acquises

Ce projet a permis de développer les compétences suivantes :

— Programmation orientée objet en Java
— Conception et modélisation UML
— Développement d'interfaces graphiques
— Gestion de bases de données relationnelles
— Implémentation de patterns de conception
— Gestion de projet logiciel

Annexe A

Script SQL de Création de la Base de Données

```sql
-- Creation de la base de donnees
CREATE DATABASE IF NOT EXISTS pharmacie_db
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE pharmacie_db;

-- Table des gestionnaires
CREATE TABLE gestionnaire (
    id_gestionnaire INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    login VARCHAR(50) UNIQUE NOT NULL,
    pwd VARCHAR(255) NOT NULL
);

-- Table des pharmaciens
CREATE TABLE pharmacien (
    id_pharmacien INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    login VARCHAR(50) UNIQUE NOT NULL,
    pwd VARCHAR(255) NOT NULL,
    matricule VARCHAR(50)
);

-- Table des clients
CREATE TABLE client (
    id_client INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    email VARCHAR(150),
    adresse TEXT,
    telephone VARCHAR(20)
);

-- Table des medicaments
CREATE TABLE medicament (
    id_medicament INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(150) NOT NULL,
    dosage VARCHAR(50),
```

```sql
    stock INT DEFAULT 0,
    prix_unitaire DECIMAL(10, 2) NOT NULL,
    seuil_alerte INT DEFAULT 10,
    description TEXT
);

-- Table des ventes
CREATE TABLE vente (
    id_vente INT PRIMARY KEY AUTO_INCREMENT,
    id_pharmacien INT,
    id_client INT,
    id_medicament INT,
    quantite INT NOT NULL,
    prix_total DECIMAL(10, 2) NOT NULL,
    date_vente TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    statut VARCHAR(20) DEFAULT 'valide',
    FOREIGN KEY (id_pharmacien) REFERENCES pharmacien(id_pharmacien),
    FOREIGN KEY (id_client) REFERENCES client(id_client),
    FOREIGN KEY (id_medicament) REFERENCES medicament(id_medicament)
);

-- Table des commandes
CREATE TABLE commande (
    id_commande INT PRIMARY KEY AUTO_INCREMENT,
    id_gestionnaire INT,
    id_medicament INT,
    quantite INT NOT NULL,
    date_commande TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    statut VARCHAR(20) DEFAULT 'en_attente',
    date_reception TIMESTAMP NULL,
    FOREIGN KEY (id_gestionnaire) REFERENCES gestionnaire(
   id_gestionnaire),
    FOREIGN KEY (id_medicament) REFERENCES medicament(id_medicament)
);

-- Table historique du stock
CREATE TABLE stock_historique (
    id_historique INT PRIMARY KEY AUTO_INCREMENT,
    id_medicament INT,
    quantite_avant INT NOT NULL,
    quantite_apres INT NOT NULL,
    type_operation VARCHAR(50),
    id_operation INT,
    date_operation TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_medicament) REFERENCES medicament(id_medicament)
);

-- Donnees de test
INSERT INTO gestionnaire (nom, prenom, login, pwd) VALUES
('Admin', 'System', 'admin', 'admin123');

INSERT INTO pharmacien (nom, prenom, login, pwd, matricule) VALUES
('Dupont', 'Jean', 'jdupont', 'pharma123', 'MAT001');

INSERT INTO client (nom, prenom, email, telephone) VALUES
('Martin', 'Pierre', 'martin@email.com', '0612345678');

```

```sql
INSERT INTO medicament (nom, dosage, stock, prix_unitaire, seuil_alerte)
    VALUES
('Doliprane', '1000mg', 100, 3.50, 20),
('Aspirine', '500mg', 50, 2.80, 15),
('Ventoline', '100ug', 5, 8.90, 10);
```

Listing A.1 – Script de création de la base de données

Annexe B

Configuration du Projet

B.1 Fichier module-info.java

```
module GestionPharmacie {
    requires java.sql;
    requires java.desktop;
    requires java.prefs;
}
```

Listing B.1 – module-info.java - Configuration du module Java

B.2 Dépendances Requises

Pour exécuter l'application, les bibliothèques suivantes sont nécessaires :

— **mysql-connector-java-8.x.jar** : Connecteur JDBC MySQL

— **JDK 11+** : Kit de développement Java

B.3 Instructions d'Installation

1. Installer MySQL Server
2. Créer la base de données avec le script SQL fourni
3. Configurer les paramètres de connexion dans `DatabaseConnection.java`
4. Ajouter le connecteur MySQL au classpath
5. Compiler et exécuter la classe `Main`